

PROFESSIONAL TRAINING MATERIAL · 2026 EDITION

DevOps & Cloud – 90-Day Learning Journey

DAY 02 · MODULE 2 · CHAPTER 2

Linux Command-Line Cheat Sheet

6 sections · 30+ commands · Practical examples

Navigation

Packages

Editors

Permissions

Networking

Power Tools

pwd

ls -a

cd /etc

grep "err"

chmod +x

sudo apt

ssh user@ip

[B] Beginner-Friendly

[T] Quick Reference

Aryashree Pritikrishna

<https://github.com/aryashreep/learn-devops-in-90-days>

[*] From Zero to Production – Session by Session [*]

Linux Command-Line Cheat Sheet

A quick reference for the most critical Linux commands you will use daily as a DevOps engineer. Every command includes **what it does** and a **practical example** you can run immediately.

1. Navigation

2. Packages

3. Editors

4. Permissions

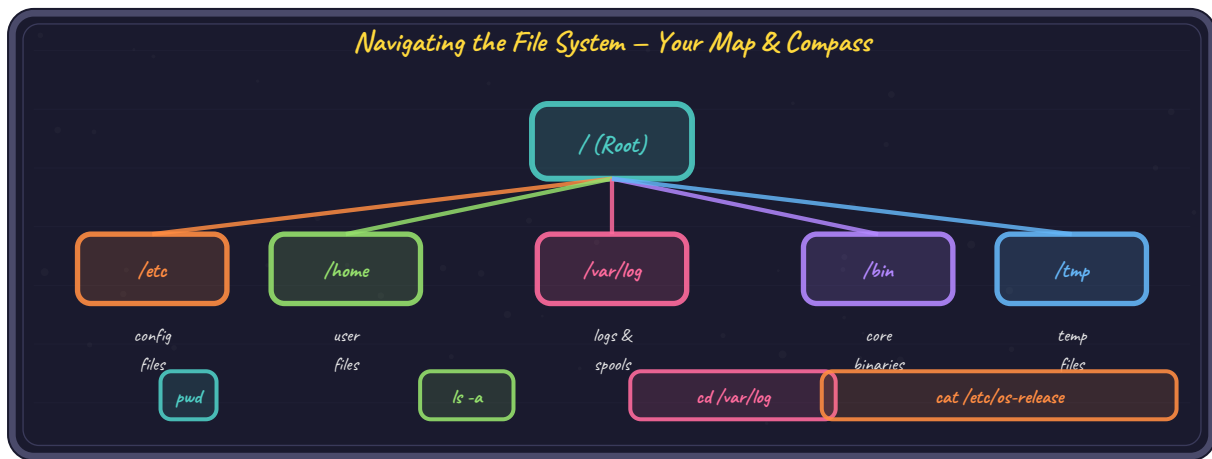
5. Networking

6. Power Tools

1

Navigating the File System & Finding Your Way

In DevOps, you navigate `/etc` for configs and `/var/log` for troubleshooting



File system map — every Linux path branches from `/` (Root). Know these paths to navigate any server confidently.

`pwd`

Prints the **Working Directory** — "Where am I right now?"

`pwd``ls -a`

Lists all files including hidden ones (starting with a dot)

`ls -a``cd <directory>`

Changes **D**irectory to the one you specify

`cd /var/log``cat /etc/os-release`

Displays the Linux distro name and version (Ubuntu, CentOS, etc.)

`cat /etc/os-release`

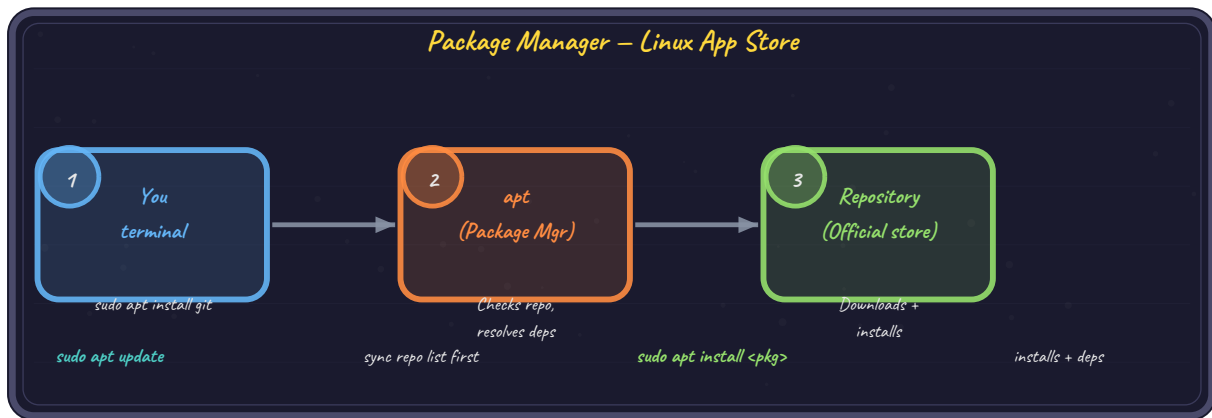
[i]

Now that you can navigate the server, the next step is managing software installed on it.

2

Managing Software (For Debian/Ubuntu)

`apt` is Linux's App Store — install from trusted repositories with one command



Package Manager flow: you request → apt checks the repo → downloads + installs with all dependencies.

```
sudo apt update
```

Refreshes the local list of available packages. **Always run this first** to avoid "not found" errors.

```
sudo apt update
```

```
sudo apt install <pkg>;
```

Downloads and installs a new piece of software plus all its dependencies.

```
sudo apt install git
```

```
sudo apt remove <pkg>;
```

Uninstalls a piece of software from the system.

```
sudo apt remove git
```



Rule: always run **sudo apt update** before any install. Without it, apt uses a stale cache and may give "package not found" or install outdated versions.

3

Viewing & Editing Files

Servers have no GUI — you must edit config files via the terminal



Nano = simple Notepad. Vim = pilot cockpit — hard to learn, but universal across every Linux system.

`nano <file>;`

Simple beginner-friendly editor.
Commands shown at bottom: **Ctrl+O** to save, **Ctrl+X** to exit.

`nano config.txt``vi <file>;`

Universal editor — two modes: press **i** for Insert Mode, **Esc** for Command Mode.

`vi config.txt``:wq (in vi)`

While in Command Mode: **writes** (saves) and **quits** the editor.

`Esc → :wq → Enter``:q! (in vi)`

Force-quits WITHOUT saving — the emergency exit when you get stuck.

`Esc → :q! → Enter``grep <text> <file>;`

Searches for a specific line of text inside a file. Perfect for finding errors in logs.

`grep "error" server.log`

Stuck in Vim? Press **Esc** (multiple times if needed) then type **:q!** then Enter. This force-quits without saving. Memorise this!

4

User & Permission Management

Control exactly who can read, write, and execute files — fundamental to Linux security



Permission string: 10 characters — file type + User (rwx) + Group (r-x) + Others (r--). Numeric: r=4, w=2, x=1. So 755 = rwxr-xr-x.

`sudo <command>;`

Superuser **do** — runs one command with root (admin) privileges. Safe and logged.

`sudo apt update`

`useradd <name>;`

Creates a new user account on the system.

`useradd newuser`

`chmod +x <file>;`

Makes a file **executable** — required step before running any script.

`chmod +x deploy.sh`

`chmod 755 <file>;`

Sets rwxr-xr-x: owner full, group/others read+execute. Standard for scripts.

`chmod 755 deploy.sh`

`chown <user>;<group>; <file>;`

Changes the **owner** and **group** that a file or directory belongs to.

`chown user`

`group`

Permission	Numeric Value	Meaning
r (Read)	4	Can view / open the file
w (Write)	2	Can modify or delete the file
x (Execute)	1	Can run the file as a program
755	7+5+5	Owner: full Group: read+exec Others: read+exec
644	6+4+4	Owner: read+write Group: read Others: read
777	7+7+7	Everyone full access — DANGEROUS, avoid in production

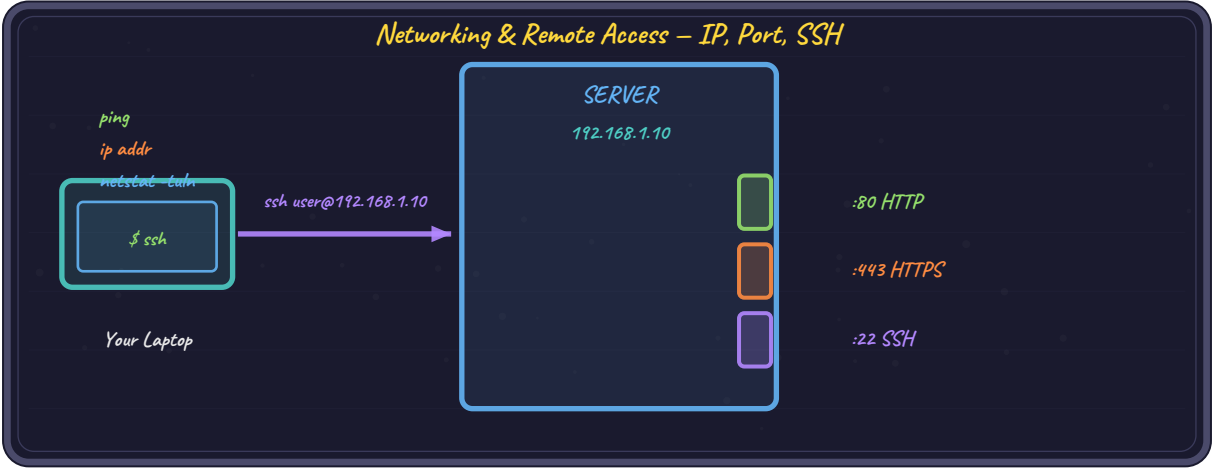


Never chmod 777 in production — it gives everyone on the system full read/write/execute access. Use 755 for scripts, 644 for config files.

5

Networking & Remote Access

Connect services, diagnose port issues, and manage servers remotely via SSH



IP = building address (find the server). Port = office number (find the service). SSH = encrypted tunnel to run commands anywhere.

`ping <destination>;`

Tests if a remote server is reachable — like asking "Are you there?"

`ping google.com`

`ip addr`

Shows the IP addresses and network interfaces of your machine. Older alternative: **ifconfig**.

`ip addr`

`netstat -tuln`

Shows all listening network ports. Perfect for checking if a web server (port 80) is running.

`netstat -tuln`

`ssh <user>@<host>;`

Initiates a **Secure Shell** connection to log into and manage a remote server over the network.

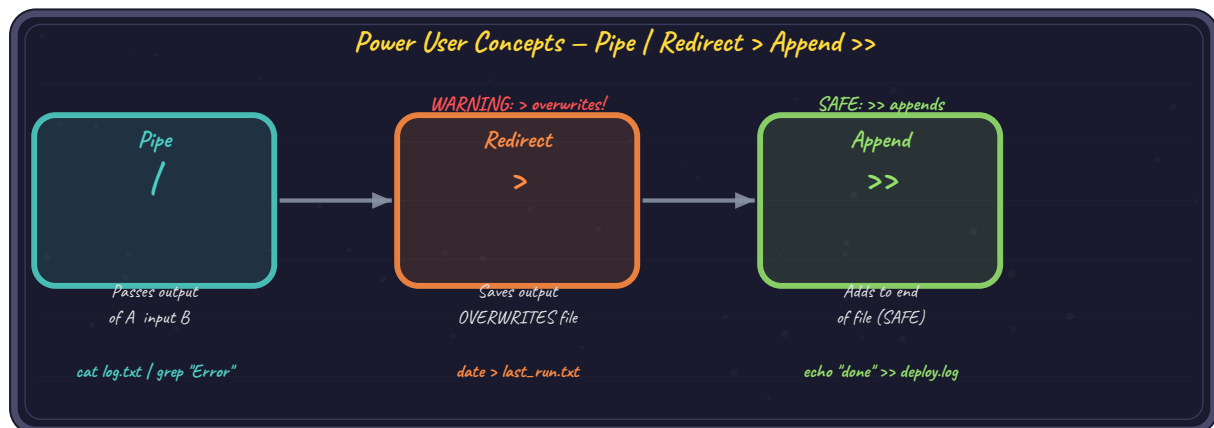
`ssh user@192.168.1.5`

Port	Service	Why It Matters
22	SSH	All remote Linux server management happens here
80	HTTP	Unencrypted web traffic — check if web server is running
443	HTTPS	Encrypted web traffic — production websites use this
3306	MySQL	Database connections — check if your DB is reachable
6443	K8s API	Kubernetes API server port — essential for kubectl

6

Power User Concepts: Combining Commands

Chain simple commands into powerful automation — the foundation of DevOps scripting



Pipe | chains commands left-to-right like an assembly line. Redirect > saves output to a file (overwrites). Append >> adds to end of file safely.

The Three Power Operators

Pipe |

```
cat log.txt | grep
"Error"
```

Find all Error lines in a log file

```
ls -la | grep ".conf"
```

List only .conf files in a directory

```
ps aux | grep nginx
```

Find if nginx process is running

```
du -h | sort -h
```

Show disk usage sorted by size

Redirect >

```
date > last_run.txt
```

Save current date to file (overwrites)

```
ls > filelist.txt
```

Save directory listing to a file

```
echo "v1.0" >
version.txt
```

Create or overwrite version file

Append >>

```
echo "Server rebooted"
>> events.log
```

Add entry to log without deleting old ones

```
date >> deploy.log
```

Append timestamp to deploy log

```
cat new.conf >>
```

```
master.conf
```

Add config to end of master file



Critical difference: > destroys all existing file content. >> adds to the end safely. When logging or journalling, always use >> to avoid data loss.

Master Quick Reference — All 30 Commands

Print this page and keep it on your desk for the first 30 days. Muscle memory for these commands is the single most valuable skill for a junior DevOps engineer.

Command	Section	What It Does
<code>pwd</code>	1. Navigation	Print Working Directory — where am I?
<code>ls -a</code>	1. Navigation	List all files including hidden (dot files)
<code>cd</code>	1. Navigation	Change Directory to specified path
<code>cat /etc/os-release</code>	1. Navigation	Show Linux distro and version info
<code>sudo apt update</code>	2. Packages	Refresh package list from repos (ALWAYS first)
<code>sudo apt install</code>	2. Packages	Install software + all dependencies
<code>sudo apt remove</code>	2. Packages	Uninstall software from system
<code>nano</code>	3. Editors	Open file in Nano — beginner friendly
<code>vi</code>	3. Editors	Open file in Vim — universal, powerful
<code>:wq (Vim)</code>	3. Editors	Save and quit Vim (from Command Mode)
<code>:q! (Vim)</code>	3. Editors	Force quit Vim without saving
<code>grep ""</code>	3. Editors	Search for text in a file
<code>sudo</code>	4. Permissions	Run one command with root privileges
<code>useradd</code>	4. Permissions	Create a new user account
<code>chmod +x</code>	4. Permissions	Add execute permission to file/script
<code>chmod 755</code>	4. Permissions	Set rwxr-xr-x — standard for scripts
<code>chown user:group</code>	4. Permissions	Change file owner and group
<code>ping</code>	5. Networking	Test if a server is reachable
<code>ip addr</code>	5. Networking	Show IP addresses and network interfaces
<code>netstat -tuln</code>	5. Networking	Show all listening ports on this machine
<code>ssh user@host</code>	5. Networking	SSH into a remote Linux server
<code>cmd1 cmd2</code>	6. Power Tools	Pipe: output of cmd1 → input of cmd2
<code>command > file</code>	6. Power Tools	Redirect: save output to file (overwrites)
<code>command >> file</code>	6. Power Tools	Append: add output to end of file (safe)
<code>tail -f</code>	6. Power Tools	Live-stream a log file (Ctrl+C to stop)
<code>cat file grep</code>	6. Power Tools	Read file and filter lines by text
<code>du -h sort -h</code>	6. Power Tools	Disk usage sorted by size — find what is big

Command	Section	What It Does
<code>history grep</code>	6. Power Tools	Search your command history
<code>ssh-keygen -t rsa</code>	6. Power Tools	Generate SSH key pair (public + private)
<code>sudo systemctl status</code>	6. Power Tools	Check if a service (nginx, ssh) is running

Practice tip: type every command from memory — do not copy-paste

Day 02 Ch.2 Complete • Next: Day 03 — Git & Version Control

<https://github.com/aryashreep/learn-devops-in-90-days> • 2026 Edition • Aryashree Pritikrishna